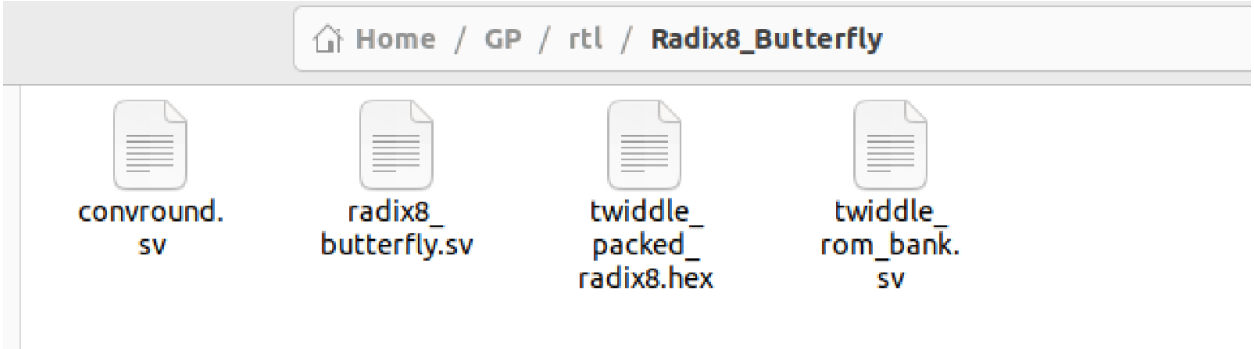
Steps:

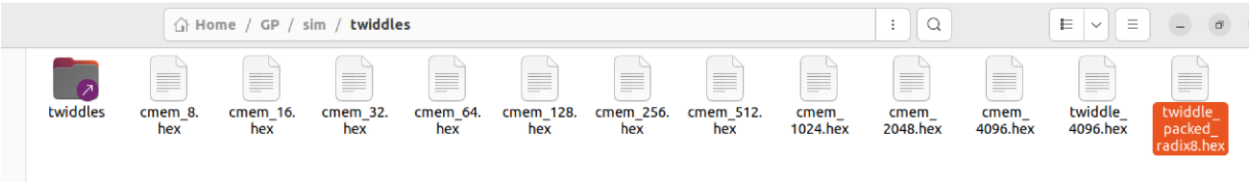
1. In GP create RTL folder paste all files



2. Adjust the path in twiddle_rom_bank.sv to where the ROM is stored for example:

```
initial begin
    $readmemh("/home/icpedia/GP/sim/twiddles/twiddle_packed_radix8.hex", rom);
end
```

So,



3. Add the. hjson file (Optimized description of the register interface).
4. Run these to generate the register file, bus interface and header (Paths may differ)

export REGGEN=$HOME/GP/.bender/git/checkouts/register_interface-1e5f22ae6b2b5b5d/vendor/lowrisc_opentitan/util/regtool.py

---

python3 $REGGEN -r rtl/Radix8_Butterfly/radix8_butterfly.hjson --outdir rtl/Radix8_Butterfly/

---

python3 $REGGEN -D rtl/Radix8_Butterfly/radix8_butterfly.hjson > pulp-runtime/include/radix8_butterfly_regs.h

---

5. Add the Wrapper
6. Modify pulp_soc.sv
7. Modify soc_interconnect_wrap.sv
8. In soc_mem_map.svh specify the address region

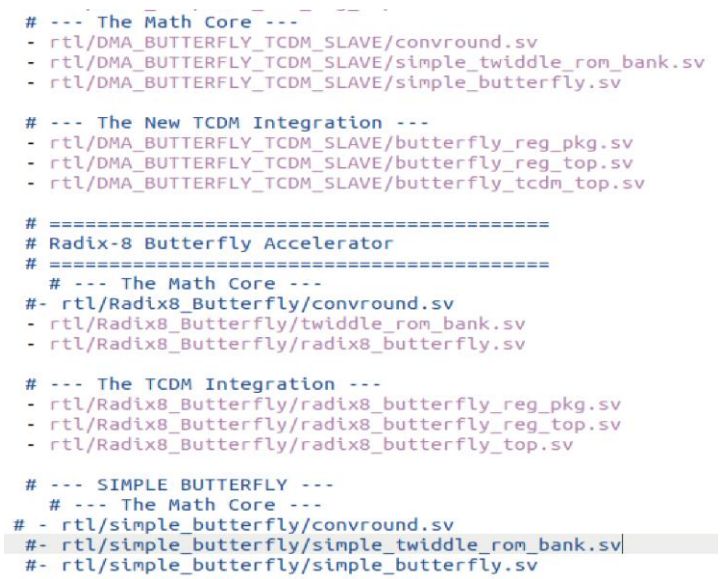
// Radix-8 Butterfly TCDM Slave

`define SOC_MEM_MAP_RADIX8_TCDM_START_ADDR 32'h1C09_1000

`define SOC_MEM_MAP_RADIX8_TCDM_END_ADDR 32'h1C09_2000

// END

9. Update bender.yml (Careful indentation)



Lines:

==

Radix-8 Butterfly Accelerator

==

--- The Math Core ---

#- rtl/Radix8_Butterfly/convround.sv

- rtl/Radix8_Butterfly/twiddle_rom_bank.sv

- rtl/Radix8_Butterfly/radix8_butterfly.sv

--- The TCDM Integration ---

- rtl/Radix8_Butterfly/radix8_butterfly_reg_pkg.sv

- rtl/Radix8_Butterfly/radix8_butterfly_reg_top.sv

- rtl/Radix8_Butterfly/radix8_butterfly_top.sv

10. Update bender and build , you can use this alias put in bashrc and source it first:

```
build_radix8_tcdm_slave() {
   echo "========================================="
   echo " [1/3] Updating Bender Manifest & Scripts"
   echo "========================================="
   cd ~/GP || return

   # CRITICAL: Forces Bender to find new/modified files in Bender.yml
   # (Uncommented because we just modified Bender.yml to fix the file collision)
   ./bender update || return

   # Generate the fresh QuestaSim compile order
   ./bender script vsim > sim/compile.tcl || return

   echo "   Applying environment fixes..."
   ./fix_bugs.sh || return 1

   echo "========================================="
   echo " [2/3] Compiling Hardware (QuestaSim)"
   echo "========================================="
   cd sim || return

   # THE NUKE AND PAVE: Destroys old compiled modules so vopt sees your new interface
   echo "   Clearing old cached modules..."
   rm -rf work/
   make clean
```

```
    echo "   Building hardware..."
    make build || return

    echo "   Running vopt optimization..."
    vopt +acc=npr -o vopt_tb tb_pulp -floatparameters+tb_pulp -work work || return

    echo "========================================="
    echo " [3/3] Hardware Build Complete!"
    echo "========================================="

    # Uncomment these when you are ready to test the C driver
    # echo "   Compiling and running TCDM C test..."
    # cd ~/GP/examples/Radix8_BUtterfly_test || return
    # make clean all
    # make run gui=1
}
```

11. Software Integration steps

Appendix:

The `reggen` tool effectively builds the bridge between your hardware module and the PULP software ecosystem. By taking the `.hjson` contract, it generates both the RTL needed to handle memory-mapped transactions and the software headers needed to target them.

Here is exactly what those three generated files represent and how you will use them to connect your `radix8_butterfly_verbose` module to the TCDM interconnect.

1. The Package File (`radix8_butterfly_reg_pkg.sv`)

This SystemVerilog package defines the data structures, parameters, and address offsets for your register map. It translates the fields from your `.hjson` into strongly-typed SystemVerilog structs.

- **`reg2hw_t` (Register to Hardware):** This struct contains the signals driven *by* the software *to* your hardware. For example, it will contain the wires for your `CTRL` register (`TRIGGER`, `CLEAR`, `CLK_EN`), your `TWIDDLE` index, and the 8 `DATA` inputs. Because we set `hwqe: "true"` on `DATA7`, this struct will also contain a `data7_qe` (Write Enable) signal.
- **`hw2reg_t` (Hardware to Register):** This struct contains the signals your hardware must drive back to the software. It will expect the wires for the `STATUS` (`DONE` flag) and the 8 `OUT` data values.
- **Constants:** It also holds local parameters for the byte offsets of each register (e.g., `RADIX8_BUTTERFLY_CTRL_OFFSET`), which is useful if you are writing testbenches.

How to use it: You will `import radix8_butterfly_reg_pkg::*;` inside your hardware wrapper module so you can declare signals of type `reg2hw_t` and `hw2reg_t`.

2. The Register Top Module (`radix8_butterfly_reg_top.sv`)

This is the actual RTL implementation of your register file and bus interface. It acts as the "shell" around your core module.

- **The Bus Interface:** It exposes a standard register bus interface (often a lightweight protocol depending on the specific `reggen` fork, usually ready to be adapted to an AXI4-Lite or generic memory-mapped bus slave port).
- **The Flip-Flops:** It instantiates the actual silicon flip-flops for the registers defined as `rw` (like `TWIDDLE` and the input `DATA` registers).
- **The Routing Logic:** For registers defined as `"hwext": "true"` (like your `OUT0`-`OUT7` and `STATUS`), it bypasses internal flip-flops and continuously routes the signals from your `hw2reg_t` input directly to the bus read data lines.
- **Address Decoding:** It handles all the address decoding, read/write enable logic, and generates error responses if the software tries to access an unmapped address.

How to use it: You will create a wrapper module. Inside this wrapper, you will instantiate `radix8_butterfly_reg_top` alongside your `radix8_butterfly_verbose` module. You connect the TCDM bus signals to the `reg_top`, and you connect the `reg2hw` and `hw2reg` struct ports between the `reg_top` and your butterfly module.

3. The C Header File (`radix8_butterfly_regs.h`)

This is the software side of the contract. It provides the bare-metal C definitions needed by the RISC-V cores to read and write your peripheral.

- **Register Offsets:** Macros defining the exact memory offset for every register (e.g., `#define RADIX8_BUTTERFLY_CTRL_REG_OFFSET 0x0`).

- **Bit Masks and Shifts:** Macros for isolating specific fields within a register. For instance, it will generate masks for your `TRIGGER` and `CLK_EN` bits, allowing software to easily perform bitwise operations without magic numbers.